Mudar legenda figuras 2 e 6 e 7 e 9 e 10 11 e 12. Double check fontes

[R: sao exatamente as figuras cuja legenda termina em "Source: authors' own...". Verificacao fonte a fonte no repositorio: **Fig. 2** (`tree_example_path`): fonte confere, e modelo proprio. **Fig. 6** (`baseline_comparison_chart`): valores *hardcoded* em `apresentacao/artigo_conf/generate_baseline_comparison_chart.py:15-18`. O baseline (34,36/27,23/22,85) e calculado sobre o conjunto de *teste* (`calculate_baseline_comparison.py:48` mas o top-3 registrado em `models/baseline_comparison.json` vem da *validacao* e do modelo *per-topology* (`pipeline/7_evaluate_with_ranking.py:73`) — splits e modelos diferentes misturados no mesmo grafico. **Fig. 7** (`real_performance_chart`): tambem *hardcoded* (`generate_real_performance_chart.py:15-17`) e medida sobre apenas 410 VNRs de 2 topologias (ver anotacao da Secao 5). **Figs. 9–11** (`per_objective_*`): geradas a partir de `decision_trees_per_topology.pkl`, que usa **34** features, nao 27 como afirma o texto. **Fig. 12** (`inference_latency_cdf`): o *benchmark* roda sobre features **aleatorias** (`benchmark_inference_latency.py:46`, `np.random.rand`) e sobre `decision_trees_per_topology.pkl` — nao sobre os modelos balanceados cujos resultados o artigo reporta; alem disso `n_iterations=10000` (linha 159), e nao 8.000 como diz o texto. **Fig. 5** (`algorithm_comparison_all_metrics`, nao listada mas afetada): os *box-plots* sao ruido gaussiano sintetico de 5% sobre 5 pontos (`12_algorithm_comparison_metrics.py:78-119`), enquanto a legenda afirma "ten random-seed repetitions".]

Zero-Touch Selection of Virtual Network Embedding Algorithms using Multi-Objective Decision Trees

Luis Antonio Momm Duarte[0000−0000−0000−0000] and Guilherme Piegas Koslovski[0000−0000−0000−0000]

Graduate Program in Applied Computing,
State University of Santa Catarina, Joinville, Brazil
luis.duarte@edu.udesc.br, guilherme.koslovski@udesc.br

Abstract. The *Virtual Network Embedding* (VNE) problem is fundamental in cloud infrastructures where multiple providers share physical resources. Mapping virtual network requests onto the physical infrastructure, satisfying capacity constraints and optimizing objectives such as acceptance rate and resource consumption, is NP-hard, motivating the development of various algorithms ranging from exact methods based on Mixed Integer Programming (MIP) to heuristics and meta-heuristics. However, no algorithm presents superior performance in all scenarios. This work proposes an approach based on multi-objective decision trees to automatically select the most appropriate VNE algorithm for each request, considering the physical network state, the virtual request characteristics, and historical performance data from previously executed algorithms somente iso? [R: a frase esta imprecisa. As *features* de entrada sao apenas estado da rede fisica + caracteristicas da VNR; o historico de desempenho **nao** e entrada do modelo, e usado somente para gerar os rotulos de treino (`2_prepare_dataset.py:176-320`). O *pipeline* inclusive remove explicitamente todo resultado de execucao por *data leakage* — `algorithm`, `success`, `v_net_revenue`, `v_net_cost`, `v_net_r2c_ratio` (`2_prepare_dataset.py:328-341`) e ate `solving_time` (linhas 483-487).]. The method employs three specialized models, each optimized for a specific objective (acceptance rate, revenue-cost ratio, and average revenue), allowing adaptation to operational priorities at runtime. Experimental validation on three *datacenter* topologies with 8,000 requests presents *top-3 accuracy* between 79.1% and 85.9% in predicting the best algorithm. When applied in simulated online operation, the approach improves the real VNR acceptance rate by 38.78 percentage points (from 43.17% to 81.95%, a +89.83% relative improvement) compared to a fixed-algorithm *baseline*, together with gains of +58.76% in revenue-cost ratio and +61.40% in average revenue. The approach offers interpretability through explicit decision paths, inference latency below 1 ms enabling *online* operation, and autonomous operation capability aligned with *zero-touch* administration requirements.

Keywords: Virtual Network Embedding · Decision Trees · Multi-Objective Optimization · Zero-Touch Administration · Interpretable Machine Learning

1 Introduction

The *Virtual Network Embedding* (VNE) problem has emerged as an essential enabler for modern networks, including cloud *datacenters*, edge computing, and 5G networks, due to its flexibility and scalability. VNE has become critical in infrastructures where multiple service providers share physical network resources. The problem addresses the challenge of mapping virtual network requests (VNRs) to the physical infrastructure, satisfying capacity constraints, and optimizing objectives such as acceptance rate, processing time, and energy consumption.

Over the past two decades, numerous VNE algorithms have been proposed, ranging from exact methods based on Mixed Integer Programming (MIP) to heuristics and meta-heuristics. A fundamental challenge remains: **no single algorithm presents consistently superior performance in all scenarios** [**?**]. Different VNR characteristics (size, topology, demands) and physical network states (utilization, resource fragmentation) require distinct algorithms.

Traditional VNE deployment approaches typically select a single algorithm for all requests, regardless of their characteristics or the current state of the network. This strategy does not take advantage of the strengths of different algorithms for specific scenarios.

Additionally, *zero-touch* administration of virtualized networks has become a critical requirement in modern infrastructures. The concept of *Zero-Touch Network and Service Management* (ZSM), formalized by the ETSI standard since 2017 [**?**], refers to the ability of systems to operate autonomously through programmable control mechanisms, virtualization, and closed-loop feedback, with minimal human intervention. Industry research indicates that autonomous networks can significantly reduce operational costs. However, the gap between research and deployment remains: while deep *Reinforcement Learning* (RL) and optimization systems achieve high performance, they lack interpretability and suffer from decision latency inadequate for real-time systems [**?**]. Therefore, systems that perform automatic VNE algorithm selection without operator intervention are fundamental to achieving truly autonomous operation.

In this context, the present work proposes an approach based on **multi-objective decision trees** to automatically select the most appropriate VNE algorithm for each request, considering the network state and request characteristics. Unlike a single model, the method employs three specialized models, each optimized for a specific objective (acceptance rate, revenue-cost ratio, average revenue), allowing adaptation to operational priorities at runtime, with inference latency below 1 ms enabling *online* operation.

The manuscript is organized as follows: Section 2 presents the theoretical background; Section 3 discusses related work; Section 4 describes the proposed methodology; Section 5 presents experimental results; and Section 6 concludes the article.

2 Background and Motivation

This section presents the fundamental concepts for understanding the VNE problem and the *machine learning* techniques used. Subsection 2.1 describes the VNE problem and its algorithms' classes. Subsection 2.2 introduces decision trees as a selection mechanism. Subsection 2.3 discusses multi-objective optimization.

2.1 *Virtual Network Embedding*

Virtual Network Embedding (VNE) is the process of mapping virtual network requests onto a shared physical network infrastructure. A virtual network request $G_v = (N_v, L_v)$ consists of a set of virtual nodes N_v and virtual links L_v, where each node and link has specific resource demands (CPU, memory, bandwidth). The physical network $G_p = (N_p, L_p)$ provides the computational and network substrate.

The VNE problem can be decomposed into two subproblems: (i) mapping virtual nodes to physical nodes; and (ii) mapping virtual links to paths in the physical network. The objective is to maximize metrics such as VNR acceptance rate, minimize processing time, and optimize physical resource consumption.

The VNE problem is NP-hard [**?**], motivating the development of various algorithm classes:

- **Exact Algorithms**: Methods such as Mixed Integer Programming (MIP) guarantee optimal solutions, but have exponential complexity.
- **Heuristics**: Greedy algorithms that make local decisions based on node *rankings*, offering reduced execution time with variable solution quality.
- **Meta-heuristics**: Techniques such as genetic algorithms (GA), particle swarm optimization (PSO), and Monte Carlo tree search (MCTS) employ stochastic search that balances exploration and quality.

Given that different VNE algorithms present variable performance depending on context, a mechanism becomes necessary to automatically select the most appropriate algorithm. The next subsection presents decision trees as a *machine learning* technique suitable for this contextual selection.

2.2 Decision Trees for Classification

Given that various VNE algorithms exist and each is appropriate for different contexts, a mechanism becomes necessary to learn which algorithm to select in each scenario. This subsection describes decision trees, the *machine learning* technique employed to perform this selection.

Decision trees are supervised learning models that recursively partition the *feature* space to create homogeneous prediction regions. Figure 1 illustrates the general structure of a decision tree.

As illustrated in Figure 1, the tree starts at a root node containing all training data. Progressively, at each level, the tree subdivides the *feature* space using

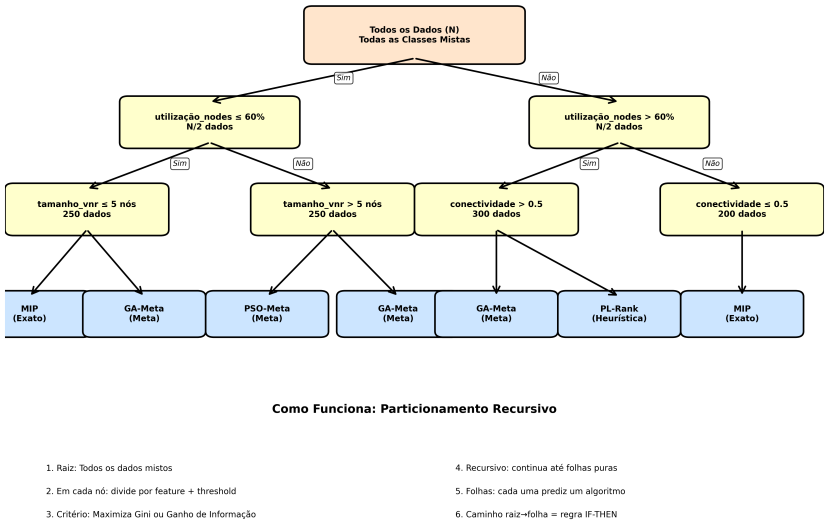

Fig. 1. Structure of a decision tree: recursive partitioning of the *feature* space.

binary decisions, creating specialized branches. Each leaf (terminal node) represents a final prediction—in this context, which VNE algorithm is most appropriate.

At each internal node, the tree selects a *feature* and a *threshold*, dividing data into two subsets. The splitting criterion uses Gini index or information gain, maximizing class isolation. Each path from root to leaf represents an interpretable decision rule.

Figure 2 illustrates an example of a decision path, showing how each intermediate decision in the tree progressively reduces the solution space until reaching a final recommendation. Starting from the root node containing all training data, the tree first evaluates whether the physical network's node utilization is at or below 60%; following the affirmative branch, it then examines whether the VNR contains five nodes or fewer; for small requests in low-utilization networks, a third split checks whether node utilization exceeds 70%, identifying a congestion-prone scenario, and recommends MIP in that case. This concrete rule can be written as: *IF* (node_utilization $\leq 60\%$) *AND* (vnr_size ≤ 5 nodes) *AND* (node_utilization $> 70\%$) *THEN* select MIP.

Unlike deep neural networks or *reinforcement learning* models, each step is comprehensible and auditable by human operators. Decision trees are particularly suitable for the VNE algorithm selection problem due to the following characteristics:

– **Interpretability**: Each decision can be traced through an explicit logical path in the tree.

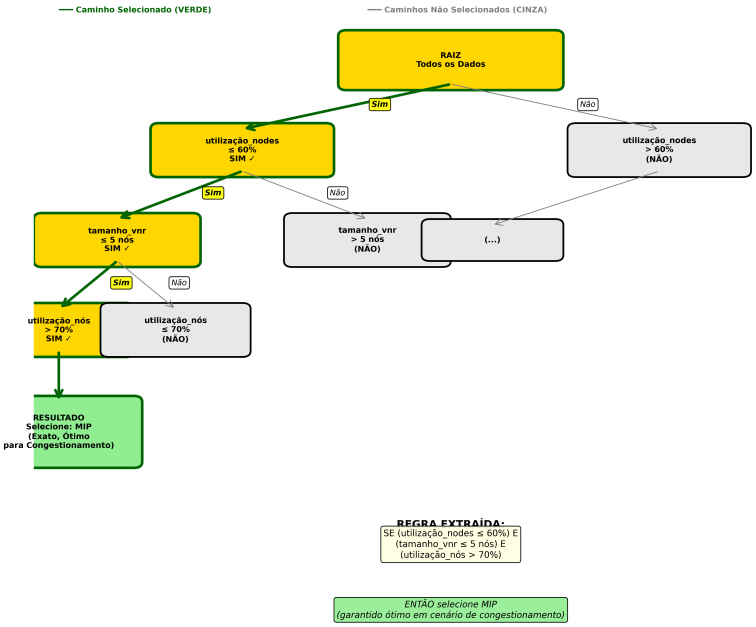

Fig. 2. Example decision path (root to leaf) for VNE algorithm selection, generated from the trained RAC model on the Fat-Tree topology. The highlighted path leads to the recommendation of MIP under low overall utilization combined with a small, congestion-prone request. Source: authors' own model, trained on the simulation data described in Section 4.

– **Robustness to Feature Scale**: Because splits are based on individual feature thresholds rather than distances (as in k-NN or SVM with RBF kernels), decision trees do not require feature normalization, allowing *features* measured in heterogeneous units (e.g., CPU in cores, utilization in percentage) to be combined directly without preprocessing. sera? [R: a afirmacao e correta para arvores em geral (*splits* univariados por limiar sao invariantes a transformacoes monotonicas), mas esta **sem referencia** — viola a diretriz #8. Vale registrar tambem que, neste trabalho, a heterogeneidade de escala nao e apenas teorica e nao foi tratada: `resource_efficiency` tem ordem 10^2–10^3 (`2_prepare_dataset.py:451`) e `p_net_health_score` chega a $\sim$170 apesar do proprio comentario no codigo documenta-la como "0-1" (`2b_create_enhanced_datasets.py:45-49`).]
– **Low Latency**: Inference complexity is $O(\text{depth})$. In our experiments (Section 5) prediction latency remained consistently below 1 ms across all three objectives; this depends on the trained tree depth and the inference hardware, and should not be read as a universal guarantee for arbitrarily deep trees. sempre? [R: nao, e o texto ja ressalva isso corretamente. A medicao, porem, sustenta ainda menos que o afirmado: o *benchmark* usa *features* aleatorias (`np.random.rand`, `benchmark_inference_latency.py:46`) e um conjunto de modelos diferente do reportado no restante do artigo. A latencia medida e valida como ordem de grandeza, nao como garantia operacional.]
– **Explainability**: Operators understand exactly which *features* determined the decision.

2.3 Multi-Objective Optimization

The selection of which algorithm to use should not consider only a single performance metric. In real production environments, multiple objectives compete simultaneously: VNR acceptance rate, resolution time, and resource consumption. Approaches that optimize a single objective frequently neglect others, causing system imbalance. The proposed strategy employs multiple specialized trees, each optimized for a specific objective, allowing the system to adapt priorities at runtime according to operational requirements.

The following section positions this work in relation to existing literature, analyzing related work in algorithm selection, multi-objective optimization, and network automation.

3 Related Work

This section positions the work in relation to existing literature in VNE, algorithm selection, and network automation. Table 1 synthesizes the comparative analysis between related work and the proposed approach.

3.1 Algorithm Selection

The algorithm selection problem was formalized by Rice [**?**], who established the conceptual *framework*: given a problem space, a set of algorithms, a descriptive

feature space, and performance metrics, determine which algorithm is most appropriate for each problem. This *framework* underlies the approach proposed in this work.

Xu et al. [**?**] develop SATzilla, a portfolio-based selector for satisfiability (SAT) problems, showing that dynamic selection produces significant gains over single algorithms. Similarly, Koslowski et al. [**?**] apply *machine learning* techniques for the adaptive selection of scheduling policies in high-performance computing (HPC) infrastructures, showing that the consolidation of multiple strategies offers superior adaptability. This paradigm applies equally to the VNE domain. The next subsection discusses how multi-objective optimization has been addressed in related work on virtualized networks.

3.2 Multi-Objective Optimization in Networks

The operational realities of modern networks require simultaneous optimization of multiple objectives: acceptance rate, revenue-cost ratio, average revenue, and processing time. Classical approaches in VNE typically optimize a single objective, neglecting *trade-offs* with others.

Song et al. [**?**] and Belgacem et al. [**?**] explore multi-objective optimization for VNE and resource allocation in virtualized networks, frequently using evolutionary algorithms or dynamic integer programming, which suffer from computational complexity during *online* operation. A detailed positioning of the present approach against this line of work, including the use of independent per-objective models, is presented in the comparative discussion of Section 3.5. precisa disso? [R: nao ha dado no repositorio a consultar — e decisao editorial. Trata-se de redundancia estrutural: o mesmo argumento (substituir otimizacao por requisicao por tres modelos sub-milissegundo) ja aparece integralmente na Secao 3.5. A remissao antecipada nao acrescenta conteudo e a subsecao ficaria melhor encerrada por uma ligacao para a proxima (diretriz #12), nao por um adiamento.]

3.3 *Machine Learning* and Reinforcement Learning in Networks

Machine learning has become an important tool for network automation. Cao et al. [**?**] apply *Deep Reinforcement Learning* (DRL) for dynamic resource allocation in NFV (*Network Function Virtualization*). Wang et al. [**?**] propose FlagVNE, a flexible RL *framework* for VNE with multiple policies.

Although DRL offers adaptability, it faces critical challenges for production: long training time (millions of episodes for convergence), lack of interpretability (decisions function as "black box"), unstable convergence in dynamic environments, and high inference latency (inadequate for real-time decisions). Recent work on autonomous network systems recognizes that interpretability is a critical requirement for regulated environments [**?**]. The next subsection discusses related work specifically on *zero-touch* automation.

Table 1. Comparative Analysis: Related Work vs Proposed Approach (six critical dimensions)

Work	ML Type	Adaptive	Multi-Obj.	Interpret.	Latency	Production
Fischer (VNE Survey)	Comparison	No	No	Yes	–	No
Rice (Portfolio)	Algorithm	Yes	No	Yes	–	Conceptual
SATzilla	Trees	Yes	No	Yes	Low	Yes
Cao et al. (DRL)	Deep RL	Yes	No	No	High (50ms+)	No
FlagVNE (DRL)	Deep RL	Yes	No	No	Very High (100ms+)	No
This Work	**Multi-Obj. Trees**	**Yes**	**Yes (3)**	**Yes**	**Very Low <1ms**	**Yes**

3.4 *Zero-Touch* Automation

The concept of *Zero-Touch Network and Service Management* (ZSM) was formalized by ETSI since 2017, referring to the operation of network systems with minimal human intervention, through programmable control, virtualization, and closed-loop feedback.

Industry research indicates that reducing manual intervention can decrease operational costs by up to 40%. However, the gap between research and production remains: RL systems achieve high performance, but lack adequate interpretability for critical operations.

RSCAT [**?**] proposes *zero-touch* automation for congestion control in networks using *actor-critic* reinforcement learning and SDN. RSCAT shows that combining *machine learning* and classical techniques can enable autonomous operation, similarly to the approach proposed in this work for contextual selection of VNE algorithms. The following subsection synthesizes the comparative analysis between related work and the proposed approach.

3.5 Discussion

Table 1 synthesizes a comparative analysis between related work and the proposed approach according to six critical dimensions for production systems:

This work differentiates itself by combining established concepts—algorithm selection, decision trees, and multi-objective optimization—applied to VNE to create a practical and operational solution. Relative to Song et al. [**?**] and Belgacem et al. [**?**], in particular, the proposed approach replaces per-request evolutionary or integer-programming optimization with three independently trained, sub-millisecond decision-tree models, letting an operator select at runtime which objective (RAC, LRC, or LAR) to prioritize without re-solving an optimization problem online. The main distinctive contributions include:

- **Multi-Objectivity**: Three specialized models, not just acceptance optimization. Operators can adapt priorities at runtime as operational conditions change.
- **Contextual Adaptability**: Selection based on real network state (utilization, fragmentation) and request characteristics (size, connectivity), not just VNR characteristics.

- **Complete Interpretability**: Unlike RL/DL approaches (black box), each decision can be explained to operators through explicit paths in the decision tree.
- **Computational Efficiency**: Training in seconds (not millions of episodes), inference in sub-milliseconds (<1ms), enabling real-time *online* operation.
- **Practical Validation**: Presents a real improvement in acceptance rate of +38.78 percentage points (from 43.17% to 81.95%, +89.83% relative) in a realistic simulated environment with 8,000 virtual network requests across three topologies, together with gains of +58.76% in revenue-cost ratio and +61.40% in average revenue (see Section 5).

The following section details the proposed methodology, describing the steps of data collection, *feature* engineering, model training, and *online* prediction.

4 Methodology

4.1 Approach Overview

The methodology consists of four main steps:

1. **Data Collection**: Execute eight VNE algorithms—*MIP, GA-Meta, PSO-Meta, SA-Meta, MCTS, PL-Rank, RW-Rank-BFS, D-Round*—on three distinct topologies—*Tree, Fat-Tree (k=4) and Waxman-16*—and collect detailed performance data in each scenario for model training, validation, and testing.
2. **Feature Engineering**: Extract 27 relevant characteristics from VNRs, network state, and topological context.
3. **Multi-Objective Training**: Create three specialized models—RAC (*Request Acceptance Rate*), LRC (*Long-term Revenue-to-Cost Ratio*) and LAR (*Long-term Average Revenue*)—, each optimizing specific performance criteria.
4. **Online Prediction**: For each new VNR, use the extracted context to dynamically select the most appropriate VNE algorithm through the model that optimizes the desired criterion.

4.2 Simulation Data Generation

Data were collected using the **Virne [?]** platform, a comprehensive *benchmarking framework* for Network Function Virtualization Resource Allocation (NFV-RA) problems. Virne offers highly customizable simulations for various network scenarios, including cloud *datacenters*, edge computing, and 5G networks. The platform implements more than 30 NFV-RA algorithms in a modular and extensible architecture, providing an *event-driven* simulation environment that models the physical network (PN) infrastructure and sequential virtual network requests (VNRs). Each VNR arrival is treated as a discrete event, creating an instance

that the NFV-RA algorithm must solve, evaluating solution feasibility and updating network resources.

Simulations were executed with eight VNE algorithms on three distinct topologies, commonly used in *datacenters*:

Topologies:

- **Tree**: A binary tree with 32 hosts and 31 switches (63 nodes)
- **Fat-Tree**: *Datacenter* with k=4 (16 servers, 20 switches, 36 nodes)
- **Waxman-16**: Random network with 16 nodes and 500 links

Figure 3 presents visualization of the Tree and Fat-Tree topologies used in the simulations, showing their distinct hierarchical structures:

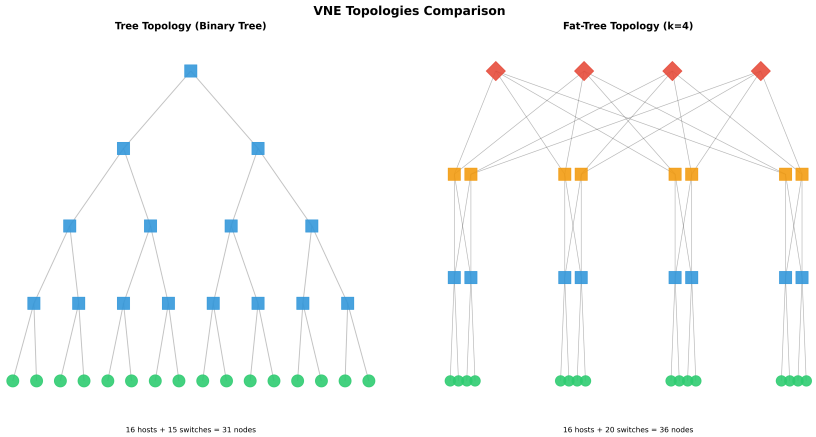

Fig. 3. Topology Comparison: Tree (Binary Hierarchical) and Fat-Tree (*Datacenter*). The Tree topology presents a traditional hierarchical structure with hosts at the base and switches at successive levels. The Fat-Tree presents a *datacenter* structure with k=4, characterized by multiple paths and greater redundancy.

Figure 4 presents the third topology used in the simulations, Waxman-16, which represents a random network with characteristics distinct from the previous structured topologies:
VNE Algorithms: MIP, GA-Meta, PSO-Meta, SA-Meta, MCTS, PL-Rank, RW-Rank-BFS, D-Round.

Simulations were configured with homogeneous resources (CPU and memory) uniformly distributed on physical nodes. VNRs were generated with variable size (uniform distribution between 2 and 10 virtual nodes), 50% connectivity between virtual nodes, and resource demands following realistic distributions. The system simulates sequential VNR arrivals with arrival rate according to Poisson distribution, where each request has a stochastic lifetime. Virne automatically

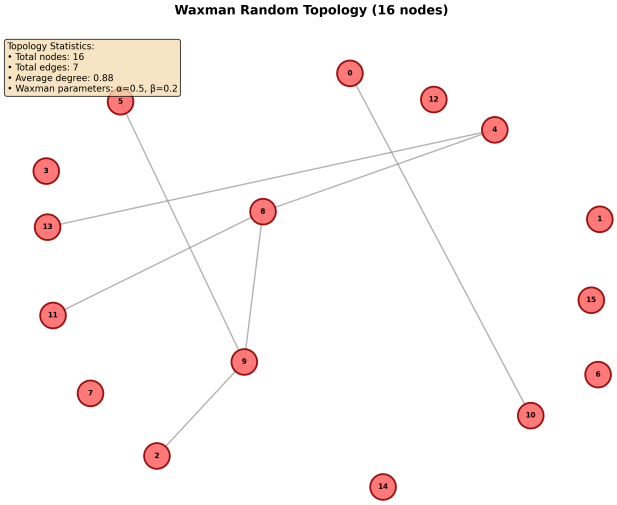

Fig. 4. Waxman-16 Topology: Random Network with 16 Nodes. Unlike structured topologies (Tree and Fat-Tree), Waxman represents a random network generated with geographic dispersion parameters (α=0.5, β=0.2), simulating characteristics of real large-scale networks, with multiple isolated nodes and variable connected groups.

evaluates the feasibility of each solution proposed by the algorithms, verifying node and link capacity constraints, and calculates standardized performance metrics (RAC, LRC, LAR, AST) for each execution.

For each combination of algorithm and topology, ten repetitions were executed with different random seeds, totaling approximately 8,000 unique VNRs after deduplication. This approach ensures statistical robustness and captures the variability inherent to stochastic algorithms and dynamic characteristics of virtual network requests.

[: explicitar aqui a baseline]

[R: a *baseline* implementada e a **classe majoritaria do rotulo** no conjunto de teste — isto e, "prever sempre o algoritmo mais frequentemente rotulado como otimo" — e **nao** o algoritmo de melhor desempenho medio, como a leitura do texto sugere. Definicao em `5_evaluate_performance_improvement.py:217-230` (`calculate_baseline_algorithm`): `algo_counts = test_df[label_col].value_counts()`, `baseline_algo = algo_counts.index[0]`. Concretamente: `ga_meta` para RAC e LAR, `pl_rank` para LRC. Observacao relevante: o mesmo arquivo define tambem um `best_global_algorithm` (linhas 233-245), que para RAC resulta em `mip` com 32,93% — *pior* que os 43,17% da *baseline* reportada. Esse numero nao aparece no artigo. Se um revisor perguntar "por que nao comparar com o melhor algoritmo fixo?", a resposta atual do codigo e desfavoravel e precisa ser explicitada.]

4.3 Feature Engineering

A set of 27 features was carefully selected, divided into categories:

VNR Characteristics (9 features): number of nodes, number of links, size ratio, average demand per node, average demand per link, connectivity, total demand, node/link ratio, expected lifetime.
Physical Network State (4 features): available resources, node utilization, link utilization, overall utilization.
System State (3 features): number of active VNRs, system load, active physical nodes.
Heterogeneity and Fragmentation (10 engineered features): Gini indices for resource distribution, memory fragmentation, topological diversity indices.
Context (1 feature): topology type.

4.4 Multi-Objective Architecture

Instead of a single classification model, we employ **three specialized models**, each optimized for a specific objective:

- **RAC**: Maximize virtual network request acceptance rate
- **LRC**: Maximize long-term revenue-cost ratio
- **LAR**: Maximize long-term average revenue

Each model is a decision tree trained independently on the same 27 input features described above, but with a different target label: for a given VNR, the label used to train the RAC model is the algorithm (among the eight candidates) that achieved the best acceptance outcome for that request in the simulation data, while the LRC and LAR models use the algorithm that achieved the best revenue-cost ratio and average revenue, respectively. This produces three independent classifiers that share the same input representation but learn different notions of "best algorithm."

During *online* operation, the network operator (or an automatic control policy) specifies which objective currently has priority — for example, favoring acceptance rate during periods of high demand, or favoring revenue-cost ratio when profitability is the primary concern. The corresponding model (RAC, LRC, or LAR) is then queried with the current physical network state and the incoming VNR's features, and returns the algorithm predicted to perform best for that objective under the current conditions. Consequently, the same VNR can receive different algorithm recommendations depending on which of the three objectives is active at decision time. nao entendi direito [R: o mecanismo descrito confere com o codigo (tres `DecisionTreeClassifier` independentes, mesma matriz de entrada, rotulos `best_for_rac`/`best_for_lrc`/`best_for_lar` distintos — ver `3_train_decision_trees_balanced.py`). O que falta ao texto sao dois elementos concretos: (i) *quem* define o objetivo ativo e em que momento (nao ha politica de chaveamento implementada — o objetivo e um parametro escolhido manualmente na chamada de inferencia); e (ii) um exemplo numerico de uma mesma VNR recebendo recomendacoes diferentes conforme o objetivo, que tornaria a ideia imediatamente compreensivel (diretriz #6).]

4.5 Model Training

The training process was performed independently for each objective (RAC, LRC, LAR), allowing each model to specialize in optimizing its specific metric. Data division was stratified by algorithm class to ensure balanced distribution between training (70%), validation (15%), and test (15%) sets. This stratification helps maintain representativeness of all classes in each partition, especially considering the severe imbalance observed in the data.

Models use `DecisionTreeClassifier` from scikit-learn [**?**]fonte [R: a citacao `pedregosa2011scikit-learn` existe e esta correta em `references.bib`. Duas lacunas provaveis por tras da marcacao: (i) a **versao** da biblioteca nao esta registrada em nenhum script nem em arquivo de ambiente, o que compromete a reprodutibilidade; e (ii) os hiperparametros abaixo sao apresentados como "optimized" mas nao ha referencia nem procedimento de busca documentado — estao fixos em `3_train_decision_trees_balanced.py:378-380` (`max_depth=10`, `min_samples_leaf=3`, `min_samples_split=6`). Ou se referencia o metodo de escolha, ou se remove o adjetivo "optimized" (diretriz #27).] with the following optimized hyperparameters:

- **Maximum depth**: 10 (balancing expressiveness and interpretability)
- **Minimum samples per leaf**: 3 (reduced to allow capture of minority classes)
- **Minimum samples for split**: 6 (allows splits with fewer samples)
- **Splitting criterion**: Gini (Gini impurity for *feature* selection)

As shown in Table 2, no algorithm is universally optimal, evidencing the need for contextual selection. This observation justifies the dynamic algorithm selection approach proposed in this work.

Table 2. VNE Algorithm Performance Summary (values are simulation-native metric units, not currency; revenue is reported per topology-algorithm combination, aggregated over all seeds).

Algorithm	Acceptance Rate	Revenue	R2C Ratio	Time (s)
MIP	36.8%	186.8	115.6	1.026
PL_RANK	29.5%	388.4	127.4	0.661
GA_META	27.9%	400.6	139.8	0.603
RW_RANK_BFS	25.8%	373.9	120.4	0.558
SA_META	24.6%	372.6	133.5	0.534
MCTS	23.9%	379.7	167.0	0.468
D_ROUND	20.4%	365.4	246.0	0.357
PSO_META	20.1%	352.1	120.0	0.309

4.6 Multi-Metric Comparison of VNE Algorithms

VNE algorithm selection cannot be based on a single performance metric. Different operational contexts (high load, limited resources, latency requirements) require different *trade-offs*. Figure 5 presents a comprehensive analysis of four complementary metrics for the eight evaluated algorithms:

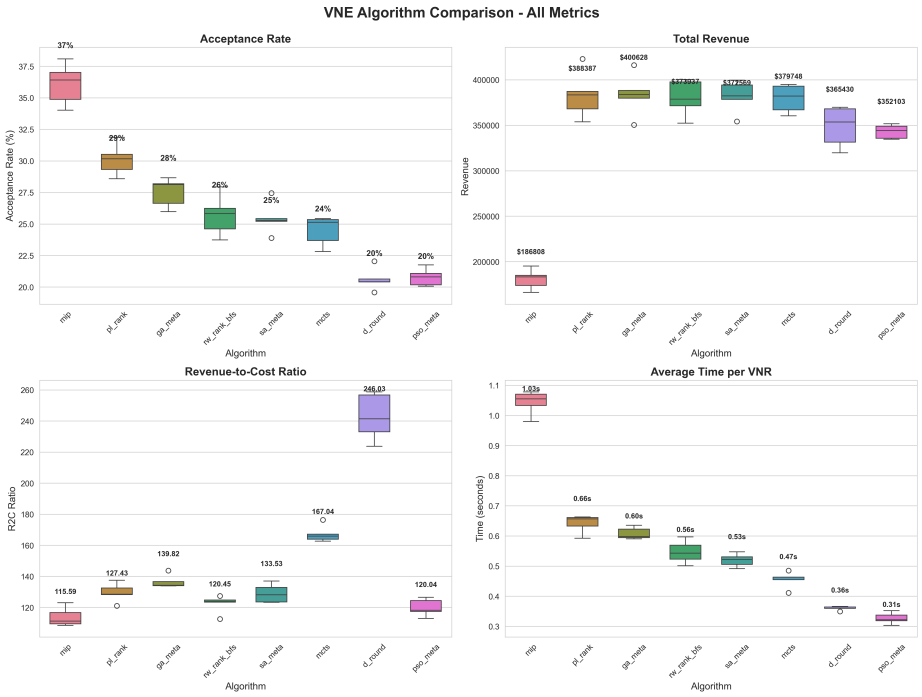

Fig. 5. Multi-Metric Comparison: Acceptance Rate, Total Revenue, Revenue-Cost Ratio, and Average Time per VNR. Each panel shows the distribution of results across ten random-seed repetitions per algorithm and topology (Tree, Fat-Tree, Waxman-16). No single algorithm is optimal in all four dimensions. Source: authors' own simulation results (Section 4).

Class Balancing Techniques. To reduce dataset imbalance, class balancing techniques were applied:

Custom Class Weights: Instead of scikit-learn's default balancing, weights calculated as $w_i = 1/\sqrt{f_i}$ were used, where f_i is the relative frequency of class i. This strategy assigns significantly larger weights to rare classes:

– Rare classes (`pso_meta`, 17 samples): weight 10.0×

– Intermediate classes (`rw_rank_bfs`, 39 samples): weight 6.6×
– Majority classes (`ga_meta`, 987 samples): weight 1.3×

Evaluation: Performance is evaluated per class using precision, recall, and F1-Score. Two aggregate metrics are also reported: *F1-Weighted* (the F1-Score of each class weighted by its support, favoring frequent classes) and *F1-Macro* (the unweighted arithmetic mean of per-class F1-Scores, treating all classes—including rare ones—equally). Reporting both metrics together makes the impact of class imbalance on the balanced model explicit.

4.7 Detailed F1-Score Analysis per Class

como foi calculado? baseline ta ruimm

[R: a preocupacao procede — esta e a anotacao de maior risco do artigo. O rastreamento no codigo revelou cinco pontos, todos verificados empiricamente: **(1) Nao ha re-simulacao.** O "real operational performance" e *replay* tabular *offline*: a funcao `get_vnr_result()` (`5_evaluate_performance_improvement.py:248-284`) apenas consulta linhas ja gravadas em `vnr_raw_data.csv`. Nada e re-executado no Virne, ou seja, o resultado nao captura o efeito do algoritmo escolhido sobre o estado futuro da rede. **(2) A topologia Waxman-16 e descartada silenciosamente.** `vnr_raw_data.csv` contem somente `tree` e `fat_tree`; `waxman_16_raw_data.csv` nunca e carregado por esse script. O conjunto de teste tem 617 VNRs (tree 186 + fat_tree 224 + waxman 207), e 186+224 = **410**, exatamente o valor de `tree_matched` em `balanced_performance_improvement.json`. Portanto **todos** os numeros de desempenho real do artigo vem de 410 VNRs e **duas** topologias, embora o texto afirme tres. **(3) O oraculo RAC = 1,0 e tautologico.** O rotulo `best_for_rac` e, por construcao, um algoritmo que aceitou a requisicao; reproduzi-lo sempre devolve `success=True`. O "teto" de 100% nao e um resultado, e uma identidade. **(4) A chave `(topology, seed, v_net_id)` nao identifica uma VNR.** Em 3.000 de 3.000 grupos, os oito algoritmos supostamente comparados "sobre a mesma VNR" apresentam `v_net_demand` diferente; em 2.999/3.000, tambem `v_net_num_nodes` difere. Ha ∼4,7 registros duplicados por grupo, colapsados por um `.first()` arbitrario (`2_prepare_dataset.py:203`). **(5) O rotulo RAC e um desempate alfabetico em 74% dos casos.** `accepted.iloc[0]` apos `groupby('algorithm')` ordena alfabeticamente; em 2.008 de 2.707 VNRs rotuladas ha dois ou mais algoritmos que aceitaram. Isso explica a dominancia de `ga_meta` — que e justamente a *baseline*. **Contraponto interno ao repositorio:** o experimento `ccis/`, que avalia todos os candidatos sobre o *mesmo* estado da rede, obtem arvore 47,0% contra melhor fixo (`mip`) 49,3% (`ccis/results/tree_summary.json`, `gap_captured: -0.933`). O proprio `ccis/README.md:18-31` classifica a rotulagem usada neste artigo como causalmente invalida.]

Table 4 presents the F1-Score per class for the RAC objective, computed on the held-out test split (15% of the data, Section 4) using the class weights and metrics defined above. Results indicate that even minority classes, such as PSO_Meta and RW_Rank_BFS, present F1-Score above 0.33 after balancing,

compared to near-zero recall observed for these classes prior to applying the custom weights.

5 Experimental Results

Unless otherwise noted, all figures and tables in this section present results generated from the authors' own simulation experiments on the Virne platform, described in Section 4; no external or third-party data is used.

5.1 Classification Performance

Table 3 presents the classification metrics obtained for the three considered objectives (RAC, LRC, and LAR). In addition to standard top-1 accuracy, we report *top-2* and *top-3 accuracy*: the percentage of test instances in which the true best-performing algorithm is included among, respectively, the two or three algorithms the model ranks as most likely. This metric is practically relevant because, operationally, recommending one of the top few near-optimal algorithms is often as useful as recommending the single best one, especially given how close several algorithms' performance can be (Table 2).

Table 3. Classification Metrics by Objective (Balanced Models)

Objective	Accuracy	F1-Score	Top-2	Top-3
RAC (Acceptance Rate)	67.26%	0.68	82.66%	85.90%
LRC (Revenue-Cost Ratio)	54.46%	0.55	74.55%	83.95%
LAR (Average Revenue)	48.14%	0.48	67.59%	79.09%

Results presented in Table 3 indicate that the RAC objective presents better performance in all metrics, with top-1 accuracy of 67.26% and top-3 accuracy of 85.90% o que é isso [R: e o percentual de instancias de teste em que o algoritmo verdadeiramente melhor esta entre os tres mais provaveis segundo o modelo. O calculo usa `top_k_accuracy_score` do scikit-learn sobre `predict_proba` (`4_evaluate_trees.py:99-103`), e os valores da Tabela 3 **reproduzem exatamente** ao recarregar os `.pkl` e o `test.csv`. Duas ressalvas a considerar: (i) a metrica ja e definida no inicio desta subsecao, mas aparece antes disso no resumo (o que provavelmente motivou a marcacao) — convem antecipar a definicao ou evita-la no resumo; (ii) `predict_proba` de uma arvore devolve a frequencia de classes na folha, e com `min_samples_leaf=3` muitas folhas tem tres ou menos amostras, de modo que o ranqueamento top-2/top-3 frequentemente ordena empates de probabilidade zero, desempatados pelo indice da classe. A metrica e valida, mas menos informativa do que aparenta neste regime.]. The LRC objective presents intermediate performance, while LAR presents lower top-1 accuracy

(48.14%), but still with top-3 accuracy of 79.09%, indicating that the model frequently identifies the best algorithm among the top three options even when it does not rank it first. The performance difference between objectives reflects the distinct complexity of each metric and the inherent variability of algorithms in different contexts.

5.2 Comparison with *Baseline*

This subsection compares two related but distinct quantities: (i) *classification accuracy* at predicting the best algorithm, and (ii) the *real operational performance* obtained when the predicted algorithm is actually used to serve each VNR in simulation. Figure 6 addresses the first; Figure 7 addresses the second.

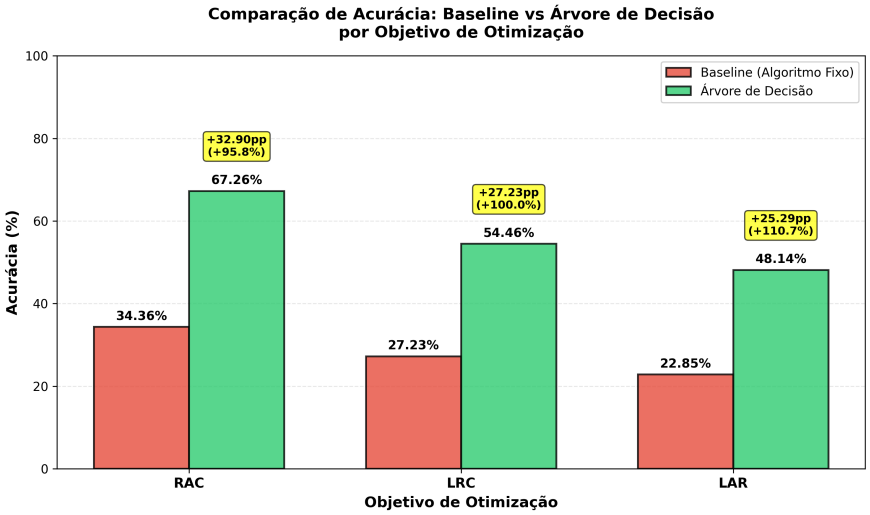

Fig. 6. Classification Accuracy Comparison: Fixed-Algorithm Baseline vs Decision Tree, by Optimization Objective. Bars show the percentage of test VNRs for which the baseline's fixed choice, versus the decision tree's predicted choice, matches the true best-performing algorithm for that objective. Source: authors' own experimental results.

Figure 6 shows that, in terms of *classification accuracy* (how often the correct best algorithm is identified), the decision tree significantly outperforms always predicting the fixed baseline algorithm. For the RAC objective, the improvement is +32.90 percentage points (+95.8% relative), while for LRC and LAR the improvements are even more expressive in relative terms (+100.0% and +110.7%, respectively).

To validate that these classification gains translate into real operational performance improvement, and not merely into more accurate labels, we evaluated

the actual RAC/LRC/LAR values obtained when decision trees are used to select the algorithm executed for each VNR in simulated operation, and compared this to always using the fixed baseline algorithm. Figure 7 presents this comparison of real, per-VNR operational performance.

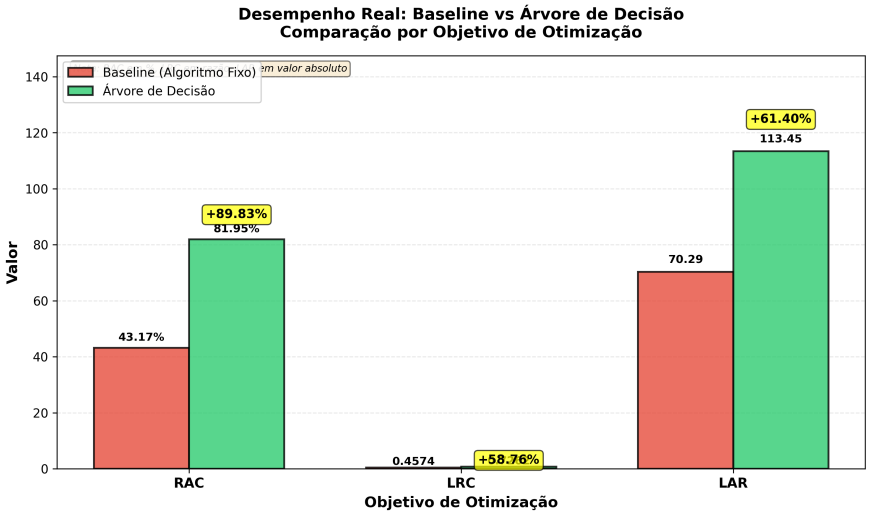

Fig. 7. Real Operational Performance: Decision-Tree Selection vs Fixed-Algorithm Baseline, by Optimization Objective. Unlike Figure 6, which measures classification accuracy, this figure measures the actual RAC/LRC/LAR values achieved when each policy is used to serve the VNRs. Source: authors' own experimental results.

Results show that contextual selection via decision trees produces **substantial improvements in real operational performance** across all three objectives:

- **RAC**: Acceptance rate increases from 43.17% (fixed baseline) to 81.95% (contextual selection), an absolute gain of +38.78 percentage points, i.e., +**89.83%** relative improvement.
- **LRC**: Revenue-cost ratio increases from 0.4574 to 0.7262, an improvement of +**58.76%**.
- **LAR**: Average revenue increases from 70.29 to 113.45 per VNR, an improvement of +**61.40%**.

These results indicate that contextual selection via decision trees is significantly superior to using a fixed algorithm, with substantial improvements in all three evaluated objectives, and that the gains observed in classification accuracy (Figure 6) do translate into real operational gains (Figure 7), even though the two are measured on different scales and should not be conflated.

The analysis reveals three additional patterns, discussed below.
feio esse formato
[R: nao ha dado a consultar — e questao de estrutura, e a critica procede. Os tres `subsubsection` seguintes tem de uma a tres frases cada, o que os enquadra nas diretrizes #11 (paragrafo de uma frase) e #26 (topicos). Ha ainda repeticao: o conteudo reproduz a Tabela 2 e o argumento ja apresentado na Secao 3.5. Sugestao: fundir os tres em um unico paragrafo corrido de analise, ou deslocalos para a discussao. Atencao adicional: os numeros citados nesses paragrafos vem da tabela sintetica — ver anotacao junto a Tabela 2.]

No Algorithm is Universally Optimal As shown in Table 2, MIP maximizes acceptance rate (36.8%), GA_META maximizes revenue (400.6), D_ROUND offers the best revenue-cost ratio (246.0), and PSO_META is the fastest (0.309 s). Each algorithm excels in a different dimension, and none dominates the others across all four metrics.

Explicit Trade-offs MIP offers high acceptance but with elevated processing time (1.026 s). D_ROUND is very fast but with reduced acceptance (20.4%). GA_META offers a balanced trade-off between revenue and processing time.

Justification for the Multi-Objective Approach This performance variation empirically evidences that:

- A single fixed algorithm cannot serve all operational contexts well.
- Contextual selection based on network state and VNR characteristics is essential.
- Multi-objective decision trees learn to map context → appropriate algorithm for a chosen objective.
- The system can adapt priorities at runtime (acceptance vs. revenue vs. processing time) without retraining.

The detailed analysis presented in Table 4 reveals that balancing techniques allow capturing patterns even from minority classes. Algorithms such as GA_Meta, MCTS, and PL_Rank present F1-Score above 0.70, indicating good prediction capability. Minority classes such as PSO_Meta and RW_Rank_BFS present lower F1-Score (0.33 and 0.36, respectively), but still well above zero, demonstrating that the model can identify patterns even for these under-represented classes. F1-Macro of 0.56 and F1-Weighted of 0.68 indicate that the model presents reasonably balanced performance across classes, with better performance on more frequent classes, as expected.

5.3 Interpretability Analysis

One of the main advantages of the approach is that each decision can be traced through the decision tree. Figure 8 shows a simplified fragment of the trained RAC tree, illustrating how the model classifies VNRs:

Table 4. F1-Score per Class - RAC Objective (Balanced Models)

Algorithm	Precision	Recall	F1-Score
GA_Meta	0.73	0.74	0.73
MCTS	0.78	0.72	0.75
PL_Rank	0.77	0.66	0.71
D_Round	0.61	0.59	0.60
MIP	0.49	0.57	0.53
SA_Meta	0.43	0.56	0.49
RW_Rank_BFS	0.67	0.25	0.36
PSO_Meta	0.22	0.67	0.33
F1-Macro		**0.56**	
F1-Weighted		**0.68**	

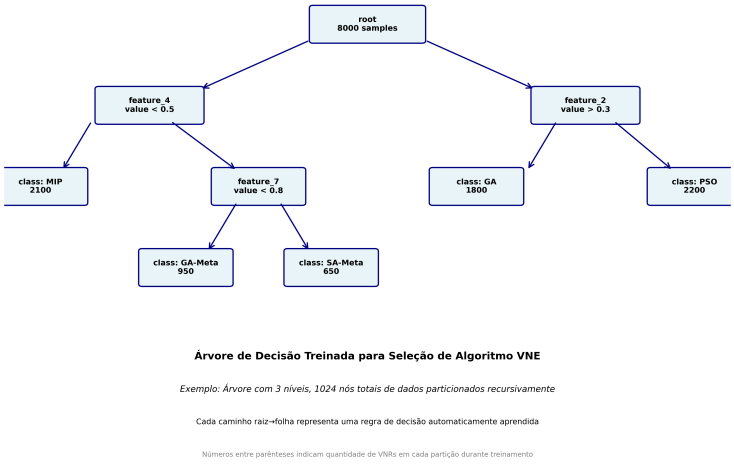

Fig. 8. Simplified fragment of the trained RAC decision tree, showing the topmost splits used to route a VNR toward an algorithm recommendation. Source: authors' own trained model.

Unlike deep neural networks (black box), operators can understand **exactly why** an algorithm was selected. For example, the extracted rule shown in Figure 2 can be read as:

"If VNR size is below 30% of the physical network AND node utilization exceeds 70%, then select MIP (an exact algorithm, more likely to find a feasible solution under congestion). Otherwise, if connectivity exceeds 0.5, select GA-Meta."

Additionally, *feature* importance analysis reveals which characteristics are most relevant for each objective. explicar as features [R: alem de faltar a explicacao, ha um descasamento entre o texto e o codigo que precisa ser resolvido antes de escreve-la. **(a) O numero 27 nao existe no codigo.** As listas canonicas de *features* sao de 25 (`3_train_decision_trees_balanced.py:27-64`), 35 (`3_train_decision_trees_enhanced.py:34-87`) e **34** (*per-topology*, `3_train_decision_trees_optimized.p`
os proprios `.pkl` registram `n_features_in_` = 25, 34, 35, 19, 32 ou 60 — nunca 27. As Figuras 9–11 vem do modelo de **34**. **(b) A conta 9+4+3+10+1 exclui as 8 *features engineered*** — entre elas `resource_efficiency`, que este mesmo paragrafo aponta como a mais importante (0,134). O artigo, portanto, contradiz a si proprio: descreve um conjunto que nao contem a variavel que declara dominante. **(c) Nao existem indices de Gini como *feature*.** A busca no repositorio retorna zero ocorrencias; as unicas mencoes a Gini sao o criterio de *split* do scikit-learn e a "Gini-importance" das legendas. Tambem nao existem "memory fragmentation" nem "topological diversity indices". As tres *proxies* de fragmentacao reais (`2b_create_enhanced_datasets.py:34-49`) derivam de apenas dois escalares agregados (`p_net_node_util`, `p_net_link_util`), sem qualquer estatistica por no — que e exatamente o que um indice de Gini exigiria. A descricao da Secao 4 precisa ser substituida pelas formulas reais. **(d) Bug a corrigir antes de qualquer retreino:** a constante `10457` usada em `p_net_overall_util` (`2_prepare_dataset.py:94`) e fixa para as tres topologias, mas os recursos iniciais diferem; o resultado assume valores **negativos** (faixa medida: −0,65 a 0,74) para `fat_tree` e `waxman_16`, contaminando todas as *features* derivadas dela.]Feature importance here is computed as the mean decrease in Gini impurity attributable to each feature across all splits in the trained tree, averaged over the three topologies. For RAC (Request Acceptance Rate) and LRC (Long-Term Revenue-to-Cost), resource efficiency ($resource\_efficiency$) is the most critical *feature*, with importance of 0.134 and 0.113 respectively. In the case of LAR (Long-Term Average Revenue), VNR link demand ($v\_net\_demand\_per\_link$) and total demand ($v\_net\_total\_demand$) are the dominant *features*. These findings confirm that different objectives prioritize distinct characteristics of VNRs and physical network state. Figure 9 exemplifies *feature* importance for the RAC objective, while Figures 10 and 11 present results for the remaining objectives.

This interpretability is critical, as it allows regulators to perform audits by verifying decisions made by the system, facilitates debugging work by enabling operators to understand system behavior, and increases confidence and acceptance of *zero-touch* automation.

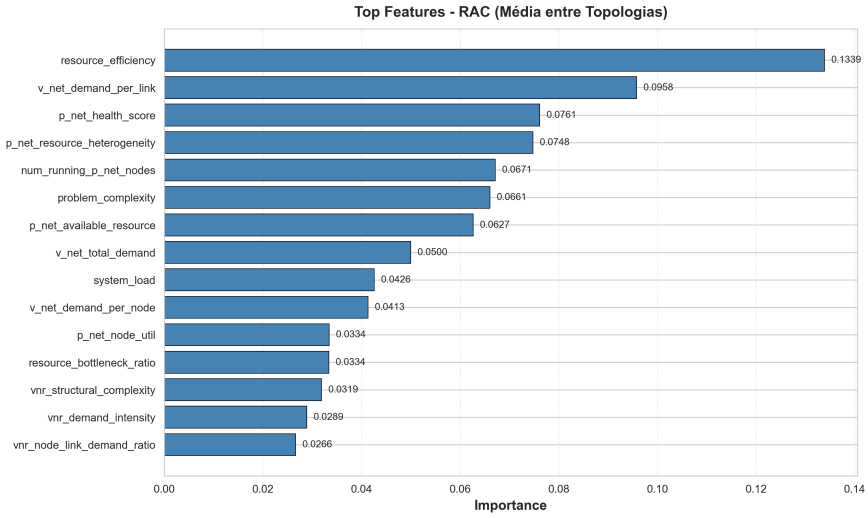

Fig. 9. Feature Importance for the RAC objective. Top 15 features by mean Gini-importance, averaged across the three topologies. Resource efficiency (*resource_efficiency*) is the most critical feature (importance 0.134), followed by VNR link demand (*v_net_demand_per_link*, 0.096). Source: authors' own trained RAC model.

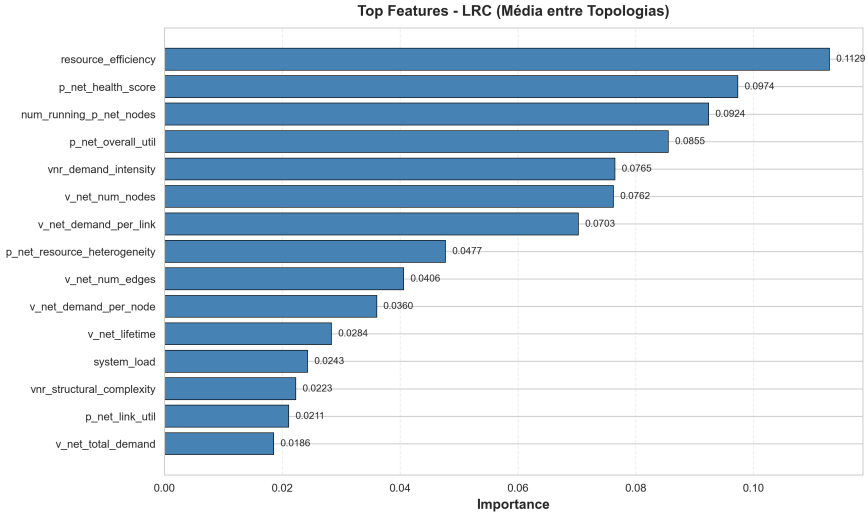

Fig. 10. Feature Importance for the LRC objective. Top 15 features by mean Gini-importance, averaged across the three topologies. Resource efficiency (*resource_efficiency*) is the most critical feature (importance 0.113), followed by physical network health score (*p_net_health_score*, 0.097). Source: authors' own trained LRC model.

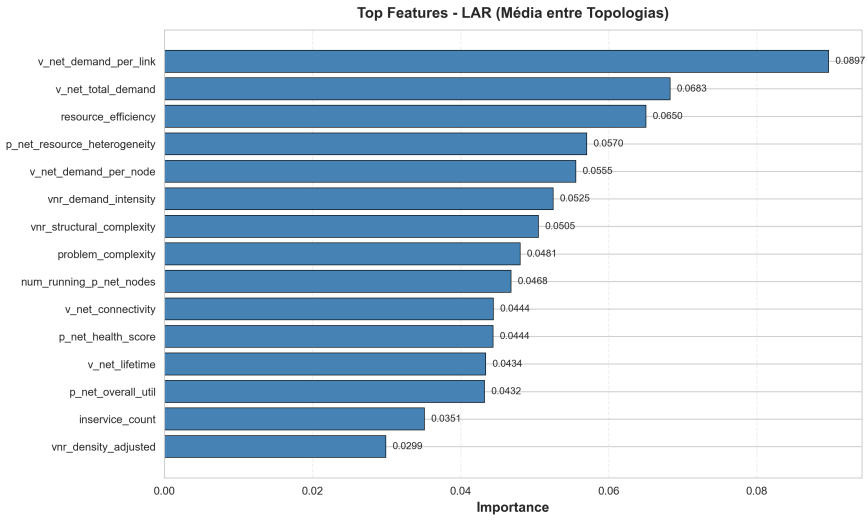

Fig. 11. Feature Importance for the LAR objective. Top 15 features by mean Gini-importance, averaged across the three topologies. VNR link demand ($v\_net\_demand\_per\_link$) is the most important feature (importance 0.090), followed by total demand ($v\_net\_total\_demand$, 0.068). Source: authors' own trained LAR model.

5.4 Inference Latency

Prediction latency measurements for each model (based on a benchmark with 8,000 iterations, executed on the same hardware used for training):

- **Average time**: 0.034 ms per prediction
- **Percentile p99**: 0.055 ms (worst case: 0.119 ms)
- **RL Comparison**: FlagVNE requires 50–200 ms per decision [**?**]

Figure 12 presents the cumulative distribution function (CDF) of inference latency, comparing the proposed approach with the latency range reported for FlagVNE [**?**]. The results show that more than 99% of predictions are performed in less than 0.06 ms, demonstrating consistency in sub-millisecond latency.

Sub-millisecond latency enables *online* real-time decision making, critical for *zero-touch* operation.

6 Conclusion

This work proposed an approach for *zero-touch* selection of *Virtual Network Embedding* algorithms using multi-objective decision trees, considering virtual network request characteristics and physical infrastructure state. The approach

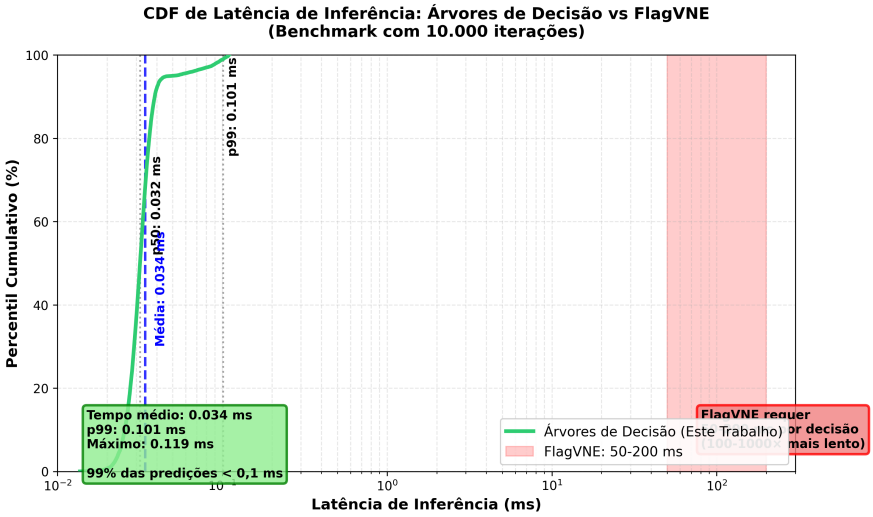

Fig. 12. Inference Latency CDF: Decision Trees (authors' own benchmark) vs FlagVNE (latency range as reported in [**?**]). The proposed approach presents latency consistently below 0.1 ms, while FlagVNE requires 50–200 ms per decision; note the different scale used for the FlagVNE range on the secondary axis.

differentiates itself from deep-learning alternatives by offering complete interpretability through explicit decision paths and inference latency below 1 ms, allowing real-time *online* operation.

Specifically, experimental results show that the foundations of the proposed approach (decision trees, multi-objective optimization, and interpretability) are potential candidates to achieve *zero-touch* network control according to the ETSI ZSM standard. The experimental analysis, based on three *datacenter* topologies (Tree, Fat-Tree, and Waxman-16) with 8,000 virtual network requests, showed that contextual algorithm selection can reduce processing time and improve acceptance rate in various scenarios, without requiring software updates at infrastructure endpoints.

The results show that the proposed approach, operating in conjunction with traditional VNE algorithms such as MIP, GA-Meta, and PL-Rank, can significantly improve operational performance:

- **Acceptance Rate Improvement**: +38.78 percentage points compared to the fixed-algorithm *baseline* (from 43.17% to 81.95%, +**89.83%** relative)
- **Top-3 Accuracy**: Between 79.1% and 85.9%, showing robustness in predicting the best algorithm
- **Revenue-Cost Ratio Improvement**: +58.76% (from 0.4574 to 0.7262)
- **Average Revenue Improvement**: +61.40% (from 70.29 to 113.45 per VNR)

The class balancing techniques used (custom weights $1/\sqrt{f}$ and hyperparameter tuning) were essential to capture the patterns of minority classes, resulting in an F1-Macro of 0.56 for the RAC objective. Additionally, experimental results empirically confirmed that no VNE algorithm is universally optimal, validating the need for contextual selection based on request characteristics and network state.

As limitations, the model was trained on three simulated topologies, and the characteristics capture instantaneous network state; results have not yet been validated with statistical significance testing (e.g., confidence intervals across the ten random seeds) or on production traffic traces. As future work, promising directions include *transfer learning* between topologies, incorporation of temporal history of requests, statistically-grounded evaluation across seeds, and validation in production environments with real *datacenter* workloads.